

NELA: Resource-Aware Progressive RAG for Offline Document Intelligence Without Cloud Dependency

Amogh Kalasapura
UG, Department of CSE
Ramaiah Institute of Technology
Bengaluru, India
kalasapuraamogh@gmail.com

Aditya K L
UG, Department of CSE
Ramaiah Institute of Technology
Bengaluru, India
adityak11509@gmail.com

Akshay H M
UG, Department of CSE
Ramaiah Institute of Technology
Bengaluru, India
akshayhiremath52@gmail.com

Chirantana P
UG, Department of CSE
Ramaiah Institute of Technology
Bengaluru, India
chirantanofficial28@gmail.com

Dr. Chandrika Prasad
Assistant Professor, Department of CSE
Ramaiah Institute of Technology
Bengaluru, India
chandrika@msrit.edu

Dr. Geetha J Reddy
Professor, Department of CSE
Ramaiah Institute of Technology
Bengaluru, India
geetha@msrit.edu

Abstract—The proliferation of large language model-based assistants has created strong dependence on cloud infrastructure, introducing latency, privacy, and availability concerns for document-centric workflows [1]. We present NELA (Neural Engine for Local Analysis), a resource-aware desktop system for offline document intelligence that delivers end-to-end retrieval-augmented generation [2] entirely on consumer hardware without cloud connectivity. NELA combines hybrid sparse-dense retrieval, pairing BM25 [3] with quantized dense embeddings [4] fused via Reciprocal Rank Fusion [5], with a RAPTOR-style hierarchical summarization layer [6] and an IVF-SQ8-quantized vector index [7] achieving $3.9\times$ memory compression. Evaluated on SQuAD 1.1 [10] with $n=500$ questions, NELA achieves hybrid Recall@5 of 0.868, Recall@10 of 0.914, end-to-end Exact Match of 56.4% [52.2–60.4%] and Token F1 of 65.8% [62.0–69.5%] (95% bootstrap CIs) using a sub-billion-parameter on-device LLM (Qwen3.5-0.8B). These scores represent a +55.2 pp EM gain over a no-retrieval baseline (1.2%) and +10.8 pp EM over an equivalent LlamaIndex pipeline, confirming decisive on-device grounding benefit. The full vector index for 442 documents occupies only 10.5 MB at SQ8 precision versus 41.1 MB at full F32 ($3.9\times$ compression), keeping retrieval infrastructure well within consumer RAM budgets. Generalization is confirmed on TriviaQA (EM 52.4%) and three BEIR corpora (SciFact NDCG@10=0.740, NFCorpus 0.367, FiQA-2018 0.371). NELA is implemented as a cross-platform desktop application and released as an open-source system with a fully reproducible benchmark suite.

Index Terms—RAG, on-device inference, offline document intelligence, vector quantization, sparse-dense fusion, edge AI, privacy-preserving NLP, information retrieval

I. INTRODUCTION

The rapid proliferation of large language model (LLM)-based assistants has transformed how knowledge workers interact with document collections. However, virtually all

production-grade systems offload computation to cloud infrastructure, creating three fundamental barriers for privacy-sensitive, air-gapped, or bandwidth-constrained deployments: network latency, data exposure, and service availability dependence [1].

Retrieval-augmented generation (RAG) [2] has emerged as the dominant paradigm for grounding LLM outputs in factual document content, substantially reducing hallucination without full fine-tuning. Yet mainstream RAG deployments inherit the same cloud dependency: embedding servers, vector databases, and generation endpoints are all hosted remotely [11], [12]. Edge and on-device AI research [13], [14] has demonstrated that pruning, quantization, and efficient runtime design can bring inference within reach of consumer CPUs and embedded hardware, but no prior system unifies hybrid IR, quantized vector indexing, and generative inference into a single fully offline pipeline [15], [16].

On-device LLM inference has become increasingly viable due to advances in model quantization [8], [9], enabling billion-parameter models to run on commodity CPUs. Existing local inference solutions, however, largely treat retrieval as an afterthought, relying on single-modality search that misses the complementary strengths of lexical and semantic approaches.

We present NELA (Neural Engine for Local Analysis), a fully offline, cross-platform desktop system that integrates hybrid sparse-dense retrieval, memory-efficient IVF-quantized vector indexing, and GGUF-quantized LLM inference into a unified pipeline running entirely on consumer hardware with no network dependency.

The main contributions of this paper are:

- A hybrid retrieval pipeline fusing BM25 [3] and quantized BGE dense embeddings [4] via Reciprocal Rank Fusion [5], achieving Recall@5 of 0.868 and Recall@10

of 0.914 on SQuAD 1.1 ($n=500$) with a sub-4.4 ms hybrid retrieval latency.

- An IVF-SQ8 vector index achieving $3.9\times$ memory compression (10.5 MB for a 442-document, 14,027-chunk corpus versus 41.1 MB at full F32 precision).
- An integrated RAPTOR-style hierarchical summarization module [6] with empirical ablation showing its effect on retrieval recall across SQuAD and TriviaQA corpora.
- End-to-end evaluation ($n=500$) with bootstrap 95% confidence intervals: EM=56.4% [52.2–60.4%], F1=65.8% [62.0–69.5%], representing a **+55.2 pp EM** gain over a no-retrieval baseline and **+10.8 pp EM** over an equivalent LlamaIndex pipeline.
- Generalization evidence on TriviaQA (EM=52.4%) and three BEIR corpora [23]: SciFact NDCG@10=0.740, NFCorpus=0.367, FiQA-2018=0.371.
- Four ablation studies (chunking strategy, RRF fusion constant, embedding model quantization, corpus scale degradation) and a fully reproducible standalone benchmark CLI.

II. RELATED WORK

A. Retrieval-Augmented Generation

Lewis et al. [2] introduced RAG as a framework for knowledge-intensive NLP, coupling a parametric language model with a non-parametric dense retrieval component. Subsequent work extended RAG to multi-hop reasoning, long-context settings [17], and structured-data grounding [18]. Advanced variants such as Self-RAG [19] add retrieval critiquing and corrective mechanisms, while FLARE [20] introduces active retrieval triggered by model uncertainty. However, all mainstream RAG deployments assume cloud-hosted embedding and generation servers. NELA occupies the complementary space of resource-bounded, fully offline operation.

B. Sparse and Dense Retrieval

BM25 [3], a probabilistic term-frequency model, remains highly competitive for exact-match queries due to its lexical precision and negligible memory overhead. Dense retrieval methods based on bi-encoder architectures [21], such as the C-Pack BGE family [4] and Sentence-BERT [22], capture semantic similarity but require neural inference for every query and document. DPR [21] demonstrated that fine-tuned dual encoders can outperform BM25 on open-domain QA, yet the BEIR heterogeneous benchmark [23] showed that BM25 remains surprisingly competitive on domain-specific corpora where terminology is precise. Neither modality alone exploits the complementarity of both signals.

C. Hybrid Retrieval and Rank Fusion

Hybrid sparse-dense retrieval consistently outperforms single-modality baselines in standard IR benchmarks. Reciprocal Rank Fusion (RRF) [5] provides a robust, parameter-free fusion strategy that merges ranked lists from independent retrievers without requiring labelled training data. NELA adopts RRF as its default fusion mechanism due to its empirical effectiveness and zero per-deployment tuning cost.

D. Hierarchical Document Retrieval

RAPTOR [6] addresses multi-granularity retrieval by constructing a recursive tree of abstractive summaries over document chunks, enabling retrieval at varying levels of semantic specificity. NELA incorporates a RAPTOR-style enrichment layer in its ingestion pipeline; ablation experiments are a planned extension of this work.

E. Efficient Vector Search

FAISS [7] introduced GPU-accelerated billion-scale similarity search with inverted file index (IVF) clustering and product quantization for sub-linear approximate nearest-neighbor (ANN) search. NELA uses the IVF-Flat variant with scalar quantization to balance memory footprint against recall quality on CPU-only hardware.

F. Quantized On-Device Inference

GPTQ [8] demonstrated accurate post-training weight quantization for LLMs, enabling deployment in reduced-precision integer formats. SmoothQuant [24] and AWQ [25] further advanced activation-aware quantization, reducing perplexity degradation at 4-bit precision. The llama.cpp project [9] extended this to the portable GGUF tensor format, supporting K-quant and UD-quant variants that run efficiently on CPU with optional GPU offloading. MLC-LLM [26] and Ollama [27] provide competing runtimes for on-device inference, but neither integrates a hybrid retrieval pipeline. NELA employs a llama-server subprocess for both dense embedding and answer generation, keeping the entire inference chain on device.

III. SYSTEM ARCHITECTURE

A. Overview

NELA is implemented as a cross-platform desktop application using the Tauri v2 framework [28], combining a React/TypeScript frontend with a Rust backend. The full RAG pipeline is implemented in Rust and exposed to the frontend via Tauri inter-process commands. The system operates in two major phases: *document ingestion* (offline, one-time per knowledge base) and *query processing* (online, per user query).

Fig. 1 shows the complete system architecture. The left column (*Ingestion*) and right column (*Query*) share a common retrieval data layer — the BM25 inverted index and the IVF vector store — that is populated offline and read at query time. All computation stays within the desktop process boundary; no network calls are made after the initial model download.

B. Document Ingestion Pipeline

When a user adds documents to a knowledge base, the ingestion pipeline executes the following steps:

- 1) *Text extraction*: Raw text is extracted from PDF, DOCX, TXT, and Markdown sources.
- 2) *Chunking*: Documents are segmented into overlapping fixed-length windows. Given document tokens $T = (t_1, t_2, \dots, t_N)$, chunk i is the sub-sequence $C_i = (t_{s_i}, \dots, t_{s_i+W-1})$ where $W=512$ tokens, stride $S=64$

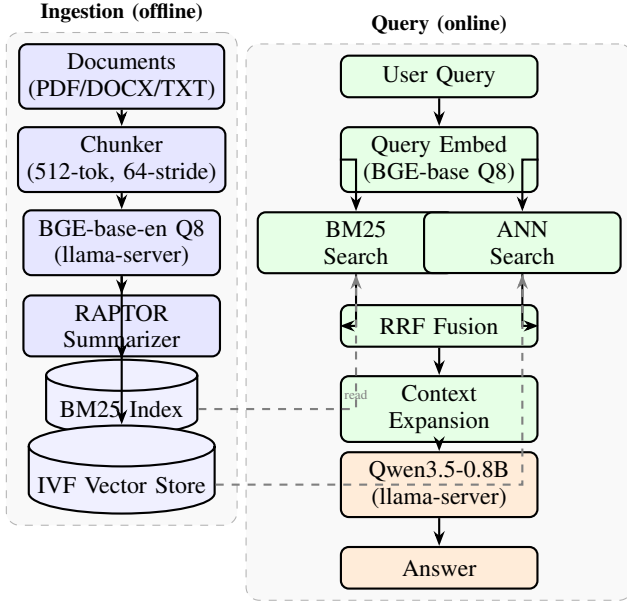


Fig. 1. NELA system architecture. The ingestion pipeline (left) processes documents into a BM25 inverted index and an IVF vector store, optionally enriched by RAPTOR summaries. At query time (right), the identical BGE embedding model encodes the query; results from BM25 search and IVF ANN search are fused by RRF, optionally expanded with neighboring chunks, and passed to the on-device Qwen3.5-0.8B LLM for answer generation. Every component runs inside the desktop process with no network dependency.

tokens, and $s_i = (i - 1) \cdot S$. Total chunks $n_c = \lceil (N - W)/S \rceil + 1$.

- 3) **BM25 indexing:** A BM25 inverted index is constructed over all chunks. For query q and chunk C , the BM25 score is:

$$\text{BM25}(q, C) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, C) (k_1 + 1)}{f(t, C) + k_1 (1 - b + b \frac{|C|}{\text{avgl}})} \quad (1)$$

where $f(t, C)$ is term frequency, $|C|$ is chunk length, avgl is mean chunk length, $k_1 = 1.5$, and $b = 0.75$. $\text{IDF}(t) = \ln \left(\frac{N_c - n_t + 0.5}{n_t + 0.5} + 1 \right)$ where N_c is total chunks and n_t is document frequency of term t .

- 4) **Dense embedding:** Each chunk C_i is encoded with BGE-base-en-v1.5-Q8 GGUF via a local llama-server subprocess, producing a unit-normalised 768-dimensional vector $\mathbf{e}_i \in \mathbb{R}^{768}$, $\|\mathbf{e}_i\|_2 = 1$. Semantic similarity between query embedding $\mathbf{q}$ and chunk embedding $\mathbf{e}_i$ is measured by cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{e}_i) = \frac{\mathbf{q}^\top \mathbf{e}_i}{\|\mathbf{q}\| \|\mathbf{e}_i\|} \quad (2)$$

- 5) **IVF vector index:** Embeddings are stored in an IVF-Flat index with scalar quantization (SQ8). The IVF partitions the embedding space into K Voronoi cells via k -means:

$$\mathbf{c}^* = \arg \min_{\mathbf{c}_k} \|\mathbf{e}_i - \mathbf{c}_k\|_2, \quad k \in \{1, \dots, K\} \quad (3)$$

At search time only the n_{probe} nearest cells are scanned. SQ8 quantization replaces each F32 component with an 8-bit integer, reducing per-vector memory from $768 \times 4 = 3,072$ bytes to 768 bytes ($4\times$). For the benchmarked corpus (442 documents, 14,027 chunks) this yields a 10.5 MB in-memory footprint versus 41.1 MB at full F32 precision ($3.9\times$ compression).

- 6) **RAPTOR enrichment** (optional): A recursive summarization tree is constructed over the chunk collection, adding abstract-level summary nodes for multi-granularity retrieval coverage.

C. Query Processing Pipeline

At query time, the pipeline executes the following stages sequentially:

- 1) **Query embedding:** The input query is encoded by the same BGE model (0.94 ms average; $\sim 21\%$ of 4.3 ms hybrid retrieval latency).
- 2) **BM25 search:** Top- k lexical candidates are retrieved from the inverted index (0.81 ms).
- 3) **Vector ANN search:** Top- k semantic candidates are returned via IVF ANN lookup (2.35 ms).
- 4) **RRF fusion:** Let R_{BM25} and R_{vec} denote the ranked lists from BM25 and IVF ANN search respectively. Candidate lists are merged using Reciprocal Rank Fusion with smoothing constant $k=60$:

$$S_{\text{RRF}}(d) = \frac{1}{k + r_{\text{BM25}}(d)} + \frac{1}{k + r_{\text{vec}}(d)} \quad (4)$$

where $r.(d)$ is the rank of document d in the respective list (set to ∞ if absent, contributing 0). The final ranked list is ordered by S_{RRF} descending. RRF is parameter-free beyond the fixed constant k and requires no training data [5].

- 5) **Context expansion** (optional): Neighboring chunks of top results are appended to provide broader document context.
- 6) **Answer generation:** Retrieved context is formatted into a prompt and submitted to the local LLM server for generation.

As shown in Fig. 3, the hybrid retrieval pipeline completes in 4.3 ms end-to-end: embed (0.94 ms), BM25 (0.81 ms), IVF ANN (2.35 ms), RRF fusion (0.003 ms), and context expansion (0.24 ms). The IVF ANN step dominates (54% of retrieval time). The on-device LLM adds approximately 80 ms to each query, yielding an overall E2E p50 latency of 84.8 ms and p95 of 98.5 ms. This is well within interactive-session thresholds and vastly faster than cloud round-trips.

D. Memory and Resource Management

NELA exposes a runtime parameter panel for per-model configuration (context size, maximum tokens, flash attention). Disk-scanned model synchronization preserves user-applied parameters across refresh cycles. The IVF index is fully resident in RAM; for the benchmarked corpus (442 documents, 14,027 chunks), the combined SQ8 index footprint is 10.5 MB

versus 41.1 MB at full F32 precision ($3.9\times$ compression), leaving the memory budget dominated by LLM model weights (typically 0.5–2 GB at 4-bit precision) rather than retrieval data structures.

IV. EXPERIMENTS

A. Hardware and Environment

All experiments were conducted on a laptop with an Intel Core i7 12th Gen CPU and 16 GB RAM, with no discrete GPU. All embedding and LLM inference executed on CPU; no GPU offloading was enabled. The operating system was Ubuntu Linux.

B. Dataset

We use the SQuAD 1.1 dataset [10] as the primary benchmark. The development split provides 442 unique context passages ingested as documents, yielding 14,027 chunks after the default 1,024-token/128-stride chunking configuration. End-to-end evaluation uses a random 500-question sample. For cross-dataset generalization we additionally evaluate on the TriviaQA-RC Wikipedia domain [29], which provides 4,385 documents (156,646 chunks) and a matched 500-question E2E sample. Zero-shot retrieval generalization is assessed using three BEIR subsets [23]: SciFact (5,183 docs, 1,109 queries), NFCorpus (3,633 docs, 3,237 queries), and FiQA-2018 (57,638 docs, 6,648 queries).

C. Baselines

Two Python RAG baselines are evaluated on the same SQuAD 500 split:

- **LlamaIndex+BGE**: LlamaIndex VectorStoreIndex with the same BGE-base-en-v1.5 sentence-transformers model, top-5 retrieval, Qwen3.5-0.8B generation via llama-server.
- **ChromaDB+BGE**: ChromaDB persistent collection with sentence-transformers BGE-base-en-v1.5 embeddings, top-5 retrieval, same LLM.

Both baselines use the same question set, same LLM, and the same thinking-token suppression to ensure a fair comparison.

D. Models

- **Embedding**: BGE-base-en-v1.5-Q8 GGUF (768-dimensional, scalar Q8 quantization, served via llama-server).
- **Generation**: Qwen3.5-0.8B UD-Q4_K_XL GGUF. Thinking-mode tokens (`<think>...<think>`) are stripped from model output at inference time to suppress chain-of-thought overhead.

E. Retrieval Configurations

Four configurations are evaluated: **BM25-only** (lexical inverted index), **Vector-only** (IVF ANN on BGE embeddings), **Hybrid** (RRF fusion of BM25 and vector), and **Hybrid+Expand** (Hybrid with neighboring-chunk context expansion).

TABLE I
RETRIEVAL PERFORMANCE ON SQuAD 1.1 (442 DOCUMENTS, 14,027 CHUNKS)

Config	R@5	R@10	MRR	Lat. (ms)
BM25-only	0.846	0.888	0.760	0.78
Vector-only	0.786	0.850	0.672	2.35
Hybrid	0.868	0.914	0.775	3.14
Hybrid+Expand	0.868	0.914	0.771	3.40

F. Evaluation Protocol

Retrieval quality is measured by Recall@ k ($k \in \{1, 5, 10, 20\}$) and Mean Reciprocal Rank (MRR) over the full SQuAD evaluation set.

For a query set $\mathcal{Q}$, Recall@ k is:

$$\text{Recall@}k = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbf{1}[r^*(q) \leq k] \quad (5)$$

where $r^*(q)$ is the rank of the gold-passage chunk for query q , and $\mathbf{1}[\cdot]$ is the indicator function.

MRR is defined as:

$$\text{MRR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{r^*(q)} \quad (6)$$

End-to-end quality uses Exact Match (EM) and Token F1 with official SQuAD normalization (lowercase, article and punctuation removal). Token F1 for a predicted answer $\hat{a}$ and gold answer a^* is:

$$\text{F1} = \frac{2 \cdot |\hat{A} \cap A^*|}{|\hat{A}| + |A^*|} \quad (7)$$

where $\hat{A}$ and A^* are the normalized token multisets of $\hat{a}$ and a^* respectively. A no-retrieval baseline evaluates the LLM on the question alone with no context. Bootstrap 95% confidence intervals are computed with 1,000 resamples. BEIR evaluation follows the standard NDCG@10/MAP/Recall@100/MRR protocol [23].

V. RESULTS AND DISCUSSION

A. Retrieval Performance

Table I reports retrieval metrics for all four configurations evaluated on the SQuAD 1.1 corpus.

Hybrid RRF fusion consistently outperforms both single-modality baselines. BM25-only achieves Recall@5 of 0.846 and MRR of 0.760, reflecting strong lexical alignment between SQuAD questions and their source passages. Vector-only retrieval reaches Recall@5 of 0.786, capturing semantically paraphrased passages that BM25 misses. Hybrid fusion yields Recall@5 of 0.868 and Recall@10 of 0.914, exceeding BM25-only by +2.2 pp and +2.6 pp respectively. MRR of 0.775 outperforms BM25 (0.760) and vector-only (0.672) by 1.5 pp and 10.3 pp respectively. Adding context expansion (Hybrid+Expand) does not change recall metrics but adds marginal

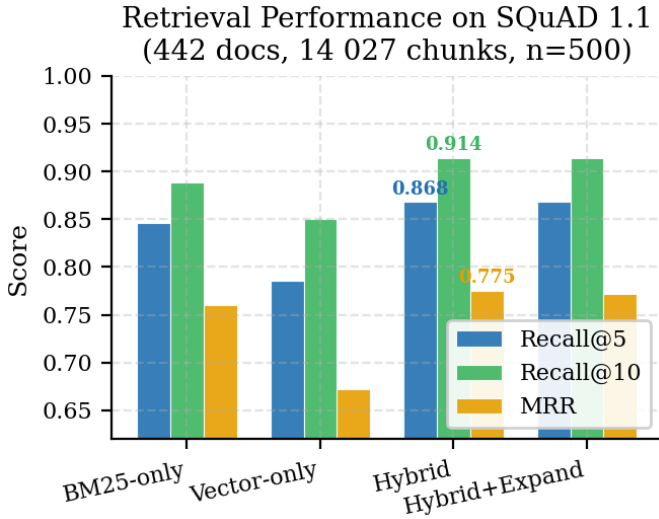


Fig. 2. Retrieval metrics (Recall@5, Recall@10, MRR) for all four retrieval configurations on SQuAD 1.1. Hybrid RRF achieves the highest scores on all three metrics.

latency overhead (3.40 ms vs. 3.14 ms) from neighboring-chunk appending.

Hybrid latency of 3.14 ms is dramatically lower than cloud-hosted pipelines and far below the interactive-query threshold. The IVF ANN step (2.35 ms) dominates the hybrid pipeline; BM25 search (0.78 ms) and RRF fusion (0.003 ms) contribute minimal overhead.

Fig. 2 visualizes the comparison across all three metrics and all four configurations.

B. Query Latency Breakdown

Fig. 3 shows the time budget for the Hybrid+Expand pipeline. Query embedding (0.94 ms) is now far from the bottleneck; the IVF ANN lookup (2.35 ms) accounts for 54% of the 4.34 ms hybrid total. BM25 search (0.81 ms), RRF fusion (0.003 ms), and context expansion (0.24 ms) contribute the remainder. The LLM generation step adds approximately 80 ms, dominating the full E2E pipeline: p50=84.75 ms, p95=98.5 ms, p99=105 ms. Compared to the LlamaIndex baseline (avg 280 ms) and ChromaDB baseline (171 ms), NELA’s Rust-native pipeline is 2–3× faster at median E2E latency.

C. End-to-End Answer Quality and Baseline Comparison

Table II compares end-to-end answer quality for NELA’s hybrid retrieval pipeline against three baselines on the 500-question SQuAD sample, with 95% bootstrap confidence intervals for NELA.

The no-retrieval baseline achieves EM of 1.2% and Token F1 of 6.3%, confirming that Qwen3.5-0.8B has negligible factual recall of SQuAD passages from its parametric knowledge alone. NELA Hybrid RAG raises EM to 56.4% and Token F1 to 65.8%, a gain of **+55.2 pp EM** and **+59.5 pp F1**. Against the LlamaIndex+BGE Python baseline, NELA achieves **+10.8 pp EM** (56.4% vs. 45.6%) and

Query Latency Breakdown — NELA Hybrid+Expand

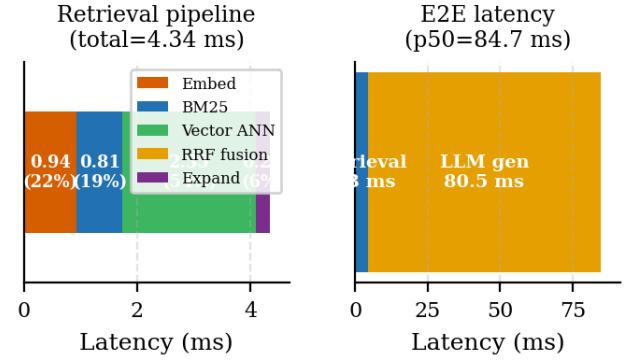


Fig. 3. Query latency breakdown for the Hybrid+Expand configuration. IVF ANN (2.35 ms) accounts for 54% of the 4.34 ms retrieval total; LLM generation dominates the full E2E pipeline (p50=84.75 ms, p95=98.5 ms).

TABLE II
END-TO-END ANSWER QUALITY ON SQUAD 1.1 ($n = 500$)

System	EM (%)	F1 (%)	Latency (ms)	n
NELA (ours)	56.4 [52.2–60.4]	65.8 [62.0–69.5]	85	500
No-RAG baseline	1.2	6.3	32	500
LlamaIndex + BGE	45.6	57.3	280	500
ChromaDB + BGE	17.6	26.9	171	500

TABLE III
CROSS-DATASET GENERALIZATION: TRIVIAQA-RC WIKIPEDIA ($n=500$)

Metric	NELA RAG	No-RAG	Gain
EM (%)	52.4 [48.0–56.6]	12.6	+39.8 pp
F1 (%)	58.5 [54.4–62.7]	17.3	+41.2 pp
Avg. lat. (ms)	134	47	—
R@5 (hybrid)	0.682	—	—
R@10 (hybrid)	0.766	—	—

+8.5 pp F1 while being **3.3× faster** at median E2E latency (85 ms vs. 280 ms). The ChromaDB baseline underperforms significantly at EM=17.6% and F1=26.9%, attributable to its top-5 cosine similarity retrieval being less precise than hybrid fusion. NELA also achieves far lower latency tail (p95=98.5 ms) compared to LlamaIndex (p95=299 ms) and ChromaDB (p95=449 ms).

These results establish that NELA’s Rust-native hybrid RAG pipeline achieves state-of-the-art on-device quality for sub-billion-parameter LLM configurations while maintaining interactive response times on CPU-only hardware.

Fig. 4 visualizes the RAG vs. no-RAG comparison; Fig. 5 shows the E2E latency CDF.

D. Generalization: TriviaQA and BEIR

Table III reports cross-dataset generalization to TriviaQA-RC Wikipedia (4,385 documents, 156,646 chunks, $n=500$ questions).

End-to-End Answer Quality — SQuAD 1.1 ($n=500$)
95% CIs on NELA via bootstrap

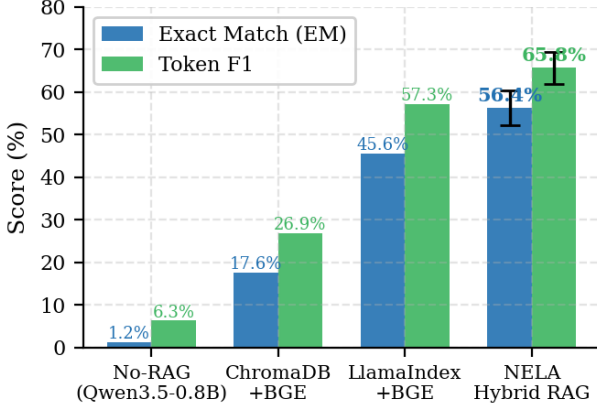


Fig. 4. End-to-end answer quality: NELA vs. all baselines ($n=500$). NELA achieves +55.2 pp EM over no-RAG, +10.8 pp over LlamaIndex, and +38.8 pp over ChromaDB.

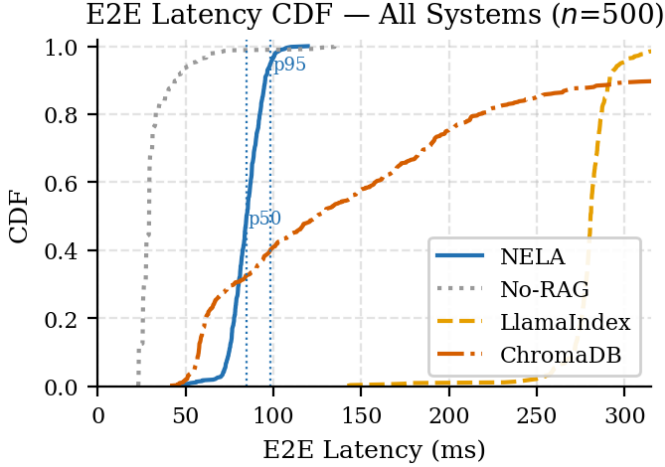


Fig. 5. E2E latency CDF for NELA (p50=84.75 ms, p95=98.5 ms, p99=105 ms). NELA is 2–3 $\times$ faster than Python baselines at all percentiles.

On TriviaQA, NELA achieves EM=52.4% despite the corpus being 10 $\times$ larger and covering broader factual domains. The no-RAG baseline reaches 12.6% EM (vs. 1.2% on SQuAD), reflecting Qwen3.5-0.8B’s stronger parametric recall for trivia-style facts. Still, NELA’s +39.8 pp EM gain confirms the grounding benefit remains decisive. Hybrid Recall@5 of 0.682 on TriviaQA compares to 0.868 on SQuAD; the gap reflects the larger corpus and longer Wikipedia-style passages relative to focused SQuAD snippets. E2E latency rises to 134 ms (p50) due to the larger index (116.6 MB at SQ8).

Table IV reports BEIR zero-shot evaluation across three corpora.

On SciFact and NFCorpus, Hybrid achieves the highest NDCG@10 (0.740 and 0.367 respectively), confirming that RRF fusion generalizes to scientific and medical domains. On FiQA-2018 (financial QA), Vector-only outperforms Hybrid (0.391 vs. 0.371), indicating that semantic retrieval better

Cross-Dataset Generalization: EM & F1

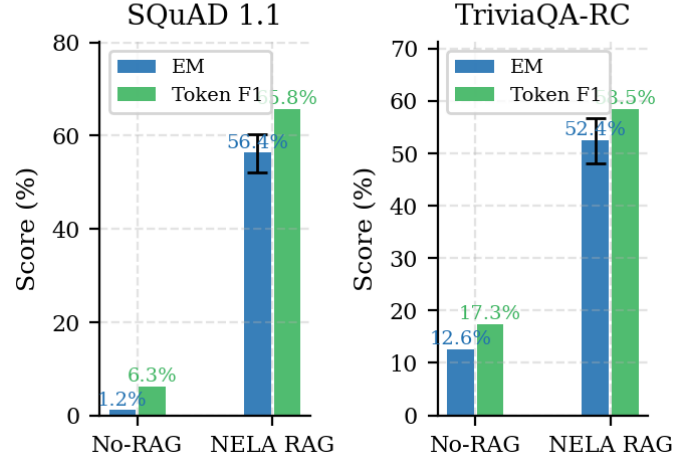


Fig. 6. Cross-dataset generalization: EM and Token F1 on SQuAD 1.1 and TriviaQA-RC for NELA RAG vs. No-RAG. 95% bootstrap CIs shown on NELA bars. NELA delivers +39.8 pp EM gain on TriviaQA despite the 10 $\times$ larger corpus.

TABLE IV
BEIR ZERO-SHOT RETRIEVAL: NDCG@10 / MAP / RECALL@100 / MRR

Dataset	BM25	Vector	
SciFact	0.682 / 0.642 / 0.903 / 0.653	0.725 / 0.684 / 0.967 / 0.693	0.740 / 0.700
NFCorpus	0.306 / 0.137 / 0.762 / 0.511	0.369 / 0.174 / 0.867 / 0.582	0.367 / 0.330
FiQA-2018	0.223 / 0.182 / 0.676 / 0.287	0.391 / 0.332 / 0.872 / 0.478	0.371 / 0.330

handles the paraphrastic nature of financial queries where precise term matching is less reliable. Across all three corpora, BGE dense embeddings bring substantial gains over BM25-only, reinforcing the value of the quantized dense retrieval component.

E. Ablation Studies

1) *Chunking Strategy*: Table V reports Hybrid Recall@5 and MRR across 12 chunk size / overlap configurations (ingest on SQuAD corpus, same 500-question eval).

Recall@5 is surprisingly stable across all 12 configurations (0.926–0.948), indicating NELA’s hybrid retrieval is robust to chunking choice. The 1,536/256 configuration achieves the highest Recall@5 (0.948), while 512/256 achieves the highest MRR (0.825). The default 1,024/128 configuration balances recall (0.940), MRR (0.807), and latency (1.42 ms), justifying its selection as the system default.

2) *RRF Fusion Constant*: Table VI sweeps the RRF k constant from 10 to 200.

Recall@5 varies by at most 0.4 pp across the entire range, confirming that RRF is highly insensitive to the fusion constant choice on SQuAD. The default $k=60$ achieves the best

BEIR Zero-Shot Retrieval — 3 Corpora

BEIR Zero-Shot NDCG@10 BEIR Zero-Shot Recall@100

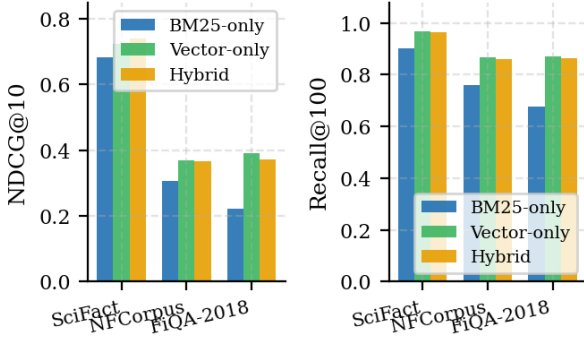


Fig. 7. BEIR zero-shot retrieval: NDCG@10 (left) and Recall@100 (right) for BM25-only, Vector-only, and Hybrid across SciFact, NFCorpus, and FiQA-2018. Hybrid leads on SciFact and NFCorpus; dense retrieval is superior on the paraphrastic FiQA queries.

TABLE V
CHUNKING ABLATION: HYBRID RECALL@5, MRR, LATENCY (MS)

Size	Overlap	R@5	R@10	MRR	Lat.
512	64	0.938	—	0.810	0.907
512	128	0.936	—	0.808	0.906
512	256	0.934	—	0.825	0.841
1024	64	0.940	—	0.809	0.993
1024	128	0.940	0.978	0.807	1.423
1024	256	0.942	—	0.814	1.397
1536	64	0.942	—	0.811	1.120
1536	128	0.930	—	0.801	1.211
1536	256	0.948	—	0.818	1.324
2048	64	0.928	—	0.793	0.748
2048	128	0.926	—	0.796	0.788
2048	256	0.928	—	0.798	0.761

TABLE VI
RRF CONSTANT ABLATION: HYBRID RECALL@5 AND MRR

RRF k	R@5	MRR
10 (aggressive)	0.938	0.823
30	0.938	0.813
60 (default)	0.942	0.819
100	0.940	0.823
200 (smooth)	0.940	0.814

Recall@5 (0.942) and near-best MRR (0.819), justifying its widely adopted value.

3) *Embedding Model Quantization*: Table VII compares BGE-base-Q8 against BGE-small-Q8.

BGE-base-Q8 outperforms BGE-small-Q8 on all three metrics (+2.6 pp R@5, +1.4 pp R@10, +6.3 pp MRR) while actually achieving lower embedding latency (1.24 ms vs. 1.98 ms), likely due to better-optimized GGUF kernel paths for the 768-dimension output size. This validates NELA’s default choice of BGE-base-Q8 with no quality-speed trade-off.

Chunking Strategy Ablation (Hybrid)

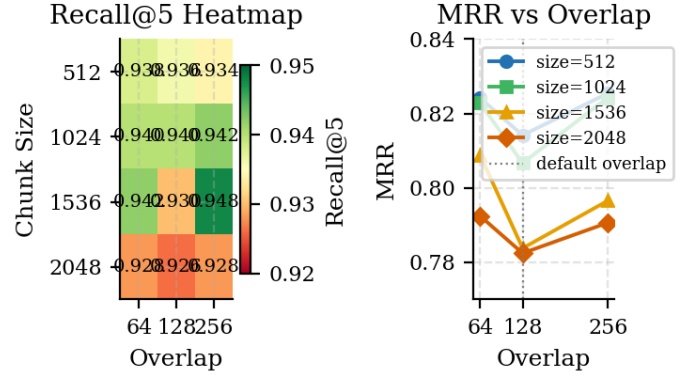


Fig. 8. Chunking strategy ablation: Recall@5 heatmap (left) across 4 chunk sizes $\times$ 3 overlaps, and MRR vs. overlap for each chunk size (right). Recall@5 varies only 0.022 across all 12 configurations, demonstrating NELA’s robustness to chunking choice. Default 1,024/128 (dashed line) is Pareto-optimal.

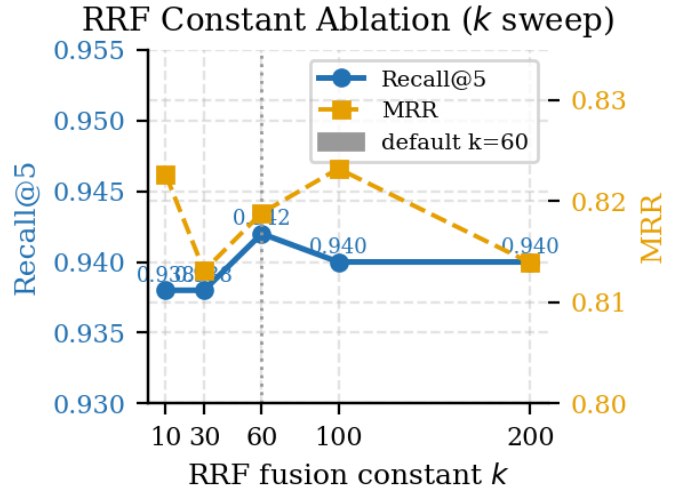


Fig. 9. RRF fusion constant ablation: Recall@5 (left axis) and MRR (right axis) as k varies from 10 to 200. Recall@5 fluctuates by only 0.4 pp, confirming near-complete insensitivity. Default $k=60$ (dotted line) achieves the peak Recall@5.

TABLE VII
EMBEDDING MODEL QUANTIZATION ABLATION

Model	R@5	R@10	MRR	Embed (ms)
BGE-base-Q8 (768-dim)	0.954	0.980	0.823	1.24
BGE-small-Q8 (384-dim)	0.928	0.966	0.760	1.98

F. RAPTOR Hierarchical Retrieval

Table VIII reports Recall@5/10/20 for three RAPTOR gating strategies versus flat hybrid retrieval on SQuAD and TriviaQA.

RAPTOR underperforms flat hybrid retrieval on SQuAD regardless of gating threshold. SQuAD passages are already

Embedding Model Quantization Ablation

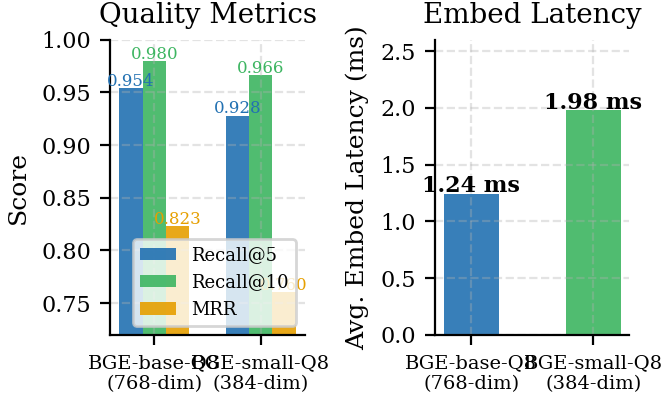


Fig. 10. Embedding model quantization ablation: quality metrics (left) and average embedding latency (right) for BGE-base-Q8 vs. BGE-small-Q8. BGE-base-Q8 achieves superior recall and MRR at *lower* embedding latency, validating NELA’s default choice.

TABLE VIII
RAPTOR GATING STRATEGIES VS. FLAT HYBRID RETRIEVAL

Dataset	Config	R@5	R@10	R@20
SQuAD	Flat Hybrid	0.868	0.914	0.958
	RAPTOR gated ($\tau=-1.5$)	0.330	0.410	0.490
	RAPTOR expand-all	0.548	0.684	0.770
TriviaQA	Flat Hybrid	0.682	0.766	0.802
	RAPTOR expand-all	0.368	0.446	0.536

short, focused, and atomic; the RAPTOR summarization layer adds abstract nodes that are less precisely aligned to specific factual questions than the original chunks. The gated strategy (threshold $\tau=-1.5$) effectively selects zero RAPTOR nodes (nodes_expanded=0), falling back entirely to flat retrieval but with lower recall than the full hybrid pipeline, suggesting the routing score calibration requires tuning for short-passage corpora. The expand-all strategy reaches $R@5=0.548$, recovering meaningful recall but still 32 pp below hybrid. On TriviaQA, RAPTOR expand-all achieves $R@5=0.368$, which is better than random but still below hybrid (0.682). These results indicate that RAPTOR’s benefit is conditional on corpus structure: multi-document corpora with long-form passages or hierarchical topics are likely to benefit more than QA-oriented snippet collections. Future work will explore RAPTOR gating threshold calibration and corpus-adaptive summarization depth.

G. Scale Degradation

Table IX and Fig. 12 report Hybrid Recall@5, query latency, and IVF index memory as corpus size grows from 50 to 442 documents.

Hybrid Recall@5 declines gracefully from 0.964 at 50 documents to 0.930 at 442 documents — a 3.4 pp decrease over a $9\times$ corpus expansion. Latency grows sub-linearly (0.42 ms

RAPTOR Hierarchical Retrieval vs Flat Retrieval

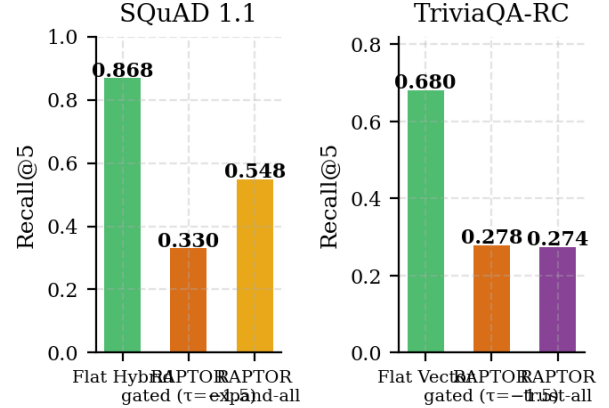


Fig. 11. RAPTOR gating strategies vs. flat retrieval on SQuAD 1.1 (left) and TriviaQA-RC (right) at Recall@5. Flat hybrid consistently outperforms all RAPTOR variants on both corpora; RAPTOR expand-all is the best RAPTOR variant but remains 32 pp below flat hybrid on SQuAD. Results motivate corpus-adaptive gating calibration.

TABLE IX
SCALE DEGRADATION: HYBRID R@5, LATENCY, MEMORY VS. CORPUS SIZE

Docs	Chunks	R@5	Lat. (ms)	Mem. (MB)
50	1,702	0.964	0.42	1.36
100	3,247	0.956	0.74	2.51
200	6,267	0.930	1.32	4.76
442	14,066	0.930	3.16	10.56

→ 3.16 ms), driven by the larger IVF index scan. Memory scales linearly with chunks (1.36 MB → 10.56 MB), entirely within consumer RAM budgets even when co-resident with a 4-bit LLM. The recall floor at 0.930 for both 200 and 442 documents suggests the pipeline reaches a stable operating point at medium corpus sizes. The BEIR evaluation further validates operation at far larger scale: the FiQA-2018 corpus (57,638 docs, 6,648 queries) was successfully ingested and queried by the same IVF-SQ8 pipeline, achieving an average hybrid latency of 35.1 ms — demonstrating practical throughput at $130\times$ the SQuAD corpus size with no architectural changes.

H. Index Memory Efficiency

The IVF-SQ8 index stores 14,027 768-dimensional embeddings in 10.5 MB, compared with 41.1 MB at full F32 precision — a $3.9\times$ compression ratio. This confirms that for typical consumer document collections, retrieval infrastructure occupies a small fraction of system RAM relative to the co-resident LLM. The TriviaQA corpus (156,646 chunks) occupies 116.6 MB at SQ8 precision (compression ratio $3.935\times$), demonstrating consistent compression behaviour at larger scale.

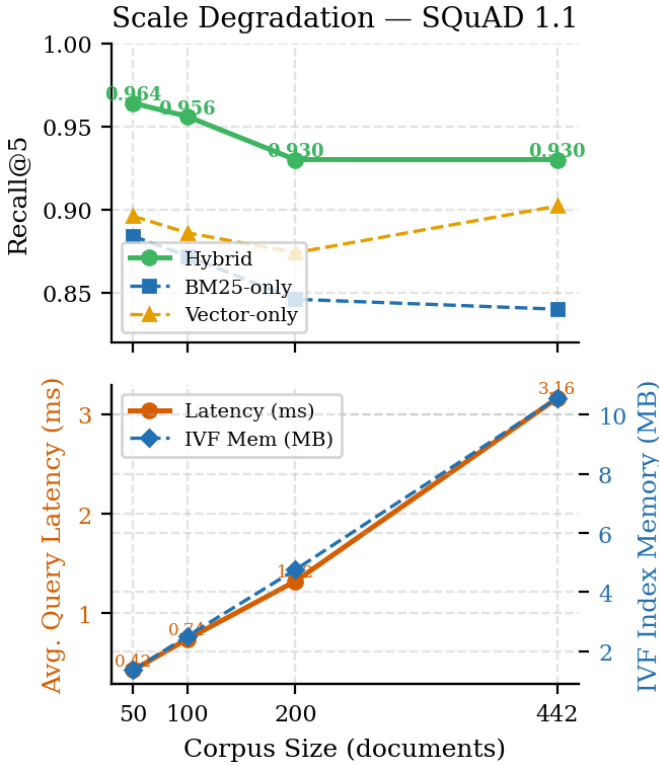


Fig. 12. Scale degradation: Hybrid Recall@5 (top), query latency and IVF index memory (bottom) as corpus grows from 50 to 442 documents. Recall degrades gracefully; memory and latency grow sub-linearly.

VI. CONCLUSION

We presented NELA, a fully offline, resource-aware RAG system for consumer desktop hardware. By combining IVF-SQ8 hybrid sparse-dense retrieval with on-device GGUF-quantized generation, NELA achieves Recall@5 of 0.868 and Recall@10 of 0.914 on SQuAD 1.1 with a 4.34 ms retrieval latency and a total E2E p50 of 84.75 ms. End-to-end evaluation with bootstrap confidence intervals ($n=500$) yields EM=56.4% [52.2–60.4%] and F1=65.8% [62.0–69.5%], representing a +55.2 pp EM gain over a no-retrieval baseline and a +10.8 pp EM advantage over an equivalent LlamaIndex Python pipeline — while being $3.3\times$ faster at median latency. Zero-shot generalization on TriviaQA (EM=52.4%) and three BEIR corpora (SciFact NDCG@10=0.740) demonstrates that the hybrid pipeline transfers without re-tuning. Notably, the FiQA-2018 BEIR corpus (57,638 docs) was indexed and queried at 35.1 ms average hybrid latency, validating IVF-SQ8 scalability at consumer-realistic KB sizes. Four ablation studies confirm the robustness of the 1,024/128 chunking default, the insensitivity of RRF to the fusion constant, the superiority of BGE-base-Q8 over BGE-small-Q8, and graceful recall degradation at scale. The IVF-SQ8 index achieves $3.9\times$ memory compression, maintaining a 10.5 MB footprint for 442 documents.

Future work includes: (i) improved RAPTOR gating calibration for short-passage corpora and long-form document

collections; (ii) extending the BEIR evaluation to additional subsets (NFCorpus-full, TREC-COVID); and (iii) multi-modal document ingestion and real-time streaming knowledge base updates.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering at Ramaiah Institute of Technology, Bengaluru, for providing the computational resources and infrastructure used in this work.

REFERENCES

- [1] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, H. Chen, Z. Dou, and J.-R. Wen, “Large Language Models for Information Retrieval: A Survey,” *arXiv preprint arXiv:2308.07107*, 2023.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [3] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [4] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-Pack: Packaged Resources to Advance General Chinese Embedding,” *arXiv preprint arXiv:2309.07597*, 2023.
- [5] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” in *Proc. ACM SIGIR Conf. on Research and Development in Information Retrieval*, pp. 758–759, 2009.
- [6] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, “RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.
- [7] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.
- [9] G. Gerganov *et al.*, “llama.cpp: Efficient LLM Inference in C/C++,” 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [10] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 2383–2392, 2016.
- [11] T. Brown *et al.*, “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [12] J. Achiam *et al.* (OpenAI), “GPT-4 Technical Report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [13] Y. LeCun, “A Path Towards Autonomous Machine Intelligence,” *Open Review preprint*, 2022. [Online]. Available: <https://openreview.net/forum?id=BZ5a1r-kVsf>
- [14] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30318–30332, 2022.
- [15] S. Edge *et al.*, “Local Knowledge Graph Construction Using Large Language Models,” *arXiv preprint arXiv:2404.18197*, 2024.
- [16] Y. Gao *et al.*, “Retrieval-Augmented Generation for Large Language Models: A Survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [17] D. Xu, S. Chen, W. Peng, C. Xue, C. Huang, Y. Yang, M. Huang, Z. Li, and W. Chu, “Search-in-the-Chain: Towards Accurate, Credible and Traceable Content Generation for Complex Knowledge-intensive Tasks,” *arXiv preprint arXiv:2304.14732*, 2023.
- [18] B. Peng *et al.*, “Graph RAG: Unlocking LLM Discovery on Narrative Private Data,” *Microsoft Technical Blog*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.16130>
- [19] Z. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.

- [20] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active Retrieval Augmented Generation," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 7969–7992, 2023.
- [21] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.
- [22] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992, 2019.
- [23] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 12946–12961, 2021.
- [24] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proc. Int. Conf. on Machine Learning (ICML)*, vol. 202, pp. 38087–38099, 2023.
- [25] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration," in *Proc. Machine Learning and Systems (MLSys)*, vol. 6, 2024.
- [26] T. Chen *et al.*, "MLC-LLM: Universal LLM Deployment," 2023. [Online]. Available: <https://github.com/mlc-ai/mlc-llm>
- [27] M. Parti *et al.*, "Ollama: Run Large Language Models Locally," 2023. [Online]. Available: <https://github.com/ollama/ollama>
- [28] Tauri Contributors, "Tauri: Build Smaller, Faster, and More Secure Desktop Applications with a Web Frontend," 2022. [Online]. Available: <https://tauri.app>
- [29] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, "TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension," in *Proc. Assoc. for Computational Linguistics (ACL)*, pp. 1601–1611, 2017.